



J-Test V2.0

Installation and User Guide

Security-hardened Functional Test Case Generator

Document	Value
Product	J-Test
Version	2.0
Guide type	Installation + End-user operation
Primary setup	Windows local development
Date	03 October 2026

Scope: This guide follows the current V2 repository: ASP.NET Core 8 backend, React 18 + Vite frontend, SQL Server LocalDB, Jira Cloud integration, Gemini 2.5 Flash, JWT authentication, and the V2 security remediations.

Contents

1. Product overview
2. Architecture and repository layout
3. Prerequisites
4. Installation and configuration
5. Starting and validating the application
6. User guide
7. Security behavior in V2
8. Troubleshooting
9. Operational checklist

Important: The repository still contains a legacy Create React App README, but package.json and the source tree show that V2 uses Vite. This guide follows the current Vite configuration rather than that legacy README.

1. Product overview

J-Test is a full-stack functional test case generator. A user can create an application account, configure Jira credentials, import Jira user stories or enter a story manually, generate functional test cases through Google Gemini, review the generated Markdown/table output, and export generated content to a Word document.

- Frontend: React 18, React Router, Material UI dependencies, Vite 8.
- Backend: ASP.NET Core 8 Web API with Entity Framework Core and JWT Bearer authentication.
- Database: SQL Server LocalDB using the J_Test_Credentials catalog.
- External services: Jira Cloud REST API and Google Gemini generateContent API.
- Protected data: application passwords are hashed; Jira API tokens are protected with ASP.NET Core Data Protection before storage.
- Output safety: AI-generated HTML is sanitized with DOMPurify before browser rendering.

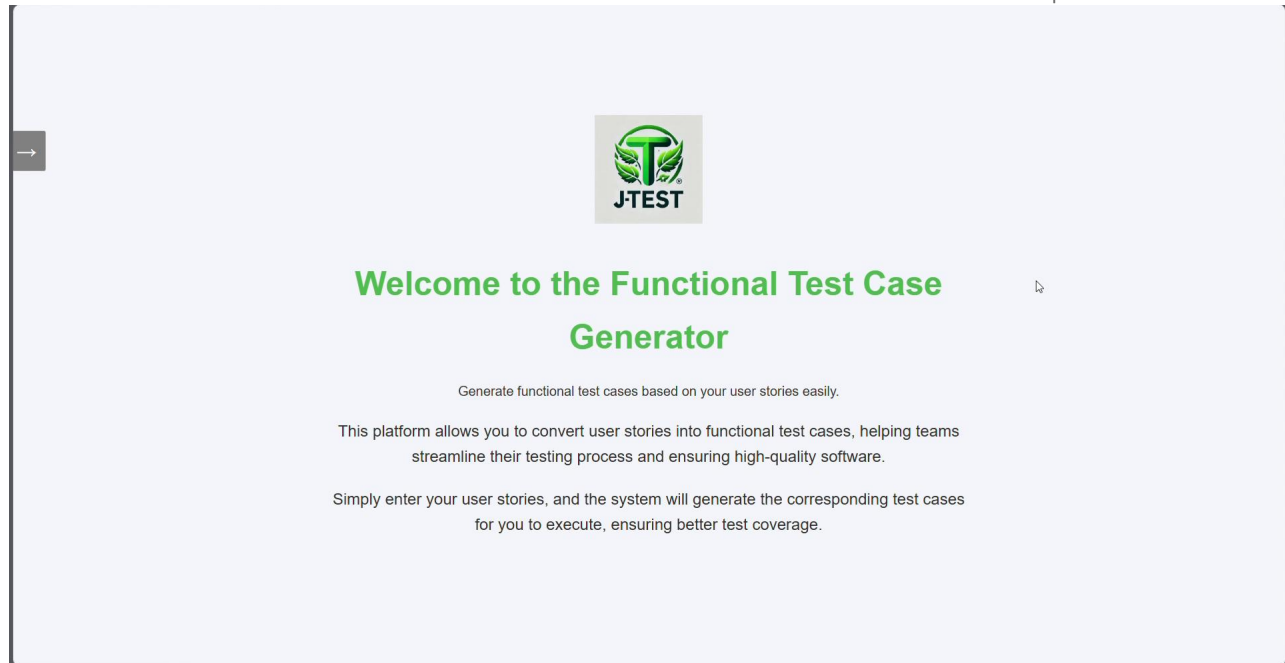
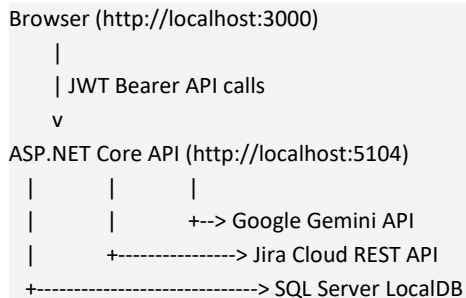


Figure 1 — J-Test home page after successful authentication

2. Architecture and repository layout

2.1 Runtime flow



2.2 Repository layout

Path	Purpose
J-Test/J-Test/	ASP.NET Core backend project
J-Test/J-Test/Controllers/	Authentication, Jira import, and Gemini generation endpoints
J-Test/J-Test/Models/	EF Core entities and request/response models
J-Test/j-test-ui/	React + Vite frontend
J-Test/Database/	Supplied LocalDB MDF/LDF database files
J-Test/Security/remediation/	V2 security remediation evidence
J-Test/J_Test_Documentations/	Architecture/design/feature documentation

J-Test/Recording/	Bundled application demonstration video
-------------------	---

3. Prerequisites

Component	Requirement / purpose
Operating system	Windows 10/11 recommended for the supplied SQL Server LocalDB setup.
.NET SDK	8.x SDK. The backend targets net8.0.
Node.js	20.19+ or 22.12+; Node 24 is also compatible with the included Vite 8.3.2 engine range.
npm	Installed with Node.js.
SQL Server LocalDB	Local database engine used by the configured connection string.
SSMS (optional)	Convenient way to attach the supplied MDF database and inspect tables.
Gemini API key	Required for AI test-case generation.
Jira Cloud account	Jira hostname, account email/username, API token, project, and issue type.
Browser	Current Edge/Chrome/Firefox.

```
dotnet --version
node -v
npm -v
SqlLocalDB info
```

4. Installation and configuration

4.1 Extract or clone the repository

Place the repository in a local development folder. The examples below assume the repository root is named J-Test.

```
cd <path-to-repository>
```

4.2 Configure SQL Server LocalDB

The backend connection string expects the LocalDB instance MSSQLLocalDB and database catalog J_Test_Credentials:

```
Data Source=(localdb)\MSSQLLocalDB;Initial Catalog=J_Test_Credentials;Integrated Security=True;TrustServerCertificate=True;
```

1. Confirm the LocalDB instance exists. If required, create/start it.
2. Connect to (localdb)\MSSQLLocalDB using SSMS or another SQL client.
3. Attach Database/J_Test_Credentials.mdf. Include the matching J_Test_Credentials_log.ldf when prompted.
4. Confirm the attached database is named J_Test_Credentials. If a different catalog name is used, update ConnectionStrings:DefaultConnection in appsettings.json.

```
SqlLocalDB info MSSQLLocalDB
SqlLocalDB start MSSQLLocalDB
```

Alternative attachment: In SSMS, use Databases > Attach > Add and select the supplied MDF. A T-SQL FOR ATTACH operation can also be used if preferred.

4.3 Configure the backend

Open a terminal in the backend folder:

```
cd J-Test\J-Test
```

Restore packages and build the project:

```
dotnet restore
dotnet build
```

4.4 Configure secrets with ASP.NET Core User Secrets

V2 intentionally does not store the Gemini API key or JWT signing key in source control. Configure both from the backend project folder:

```
dotnet user-secrets set "Gemini:ApiKey" "<YOUR_GEMINI_API_KEY>"
dotnet user-secrets set "Jwt:Key" "<STRONG_RANDOM_SECRET_AT_LEAST_32_CHARACTERS>"
dotnet user-secrets list
```

Security: Do not place real Gemini keys, JWT signing keys, Jira API tokens, or user passwords in appsettings.json, screenshots, Git commits, or documentation.

4.5 Review application configuration

The supplied appsettings.json contains the non-secret JWT issuer/audience, Gemini model, LocalDB connection string, and Jira host allowlist. The Gemini model in V2 is gemini-2.5-flash.

```
"Jwt": {
  "Issuer": "JTestApi",
  "Audience": "JTestClient"
},
"Gemini": {
  "Model": "gemini-2.5-flash"
}
```

Add the Jira Cloud hostname(s) that this installation is allowed to contact. Use hostname only—no https://, path, query string, user information, or port:

```
"Jira": {
  "AllowedHosts": [
    "yourtenant.atlassian.net"
  ]
}
```

Why this matters: The backend rejects localhost, IP literals, explicit ports, URL schemes, and hosts not present in

Jira:AllowedHosts. This is the V2 SSRF protection.

4.6 Install the frontend

Open a second terminal in the Vite frontend folder:

```
cd J-Test\j-test-ui
npm install
npm run build
```

The Vite development server is configured for port 3000. The backend CORS policy also allows <http://localhost:3000>, so keep this port unless you intentionally update both configurations.

Docker status: The repository contains Dockerfiles, but the current local V2 setup is the recommended path for this guide. The frontend Dockerfile should be reviewed before production use because the active Vite configuration and Node engine requirements have changed from the older frontend setup.

5. Starting and validating the application

5.1 Start the backend

```
cd J-Test\J-Test
dotnet run
```

With the supplied launch settings, the frontend calls the backend at <http://localhost:5104>. In Development, Swagger is enabled and mounted at the API root.

```
http://localhost:5104
```

5.2 Start the frontend

```
cd J-Test\j-test-ui
npm start
```

Open:

```
http://localhost:3000
```

Validation check	Expected result
Login page	J-Test logo and Login form are displayed.
Signup	A new account can be created and then used to log in.
Backend authentication	Successful login returns a JWT; protected API calls succeed.
Settings	Saved Jira credentials can be listed/saved/deleted.
Jira import	Stories can be fetched using a loaded credential, project, and issue type.

Gemini generation	Submitting a valid user story produces generated test cases.
Export	Export downloads Functional Test Cases.docx.
Logout	authToken is removed from sessionStorage and the UI returns to Login.

6. User guide

6.1 Create an application account

From the Login page, select Sign up. Enter an email address, password, and matching confirmation password. The backend hashes the password before saving it.

1. Open `http://localhost:3000`.
2. Select Sign up.
3. Enter Email, Password, and Confirm Password.
4. Select Sign Up.
5. After the success message, return to Login.

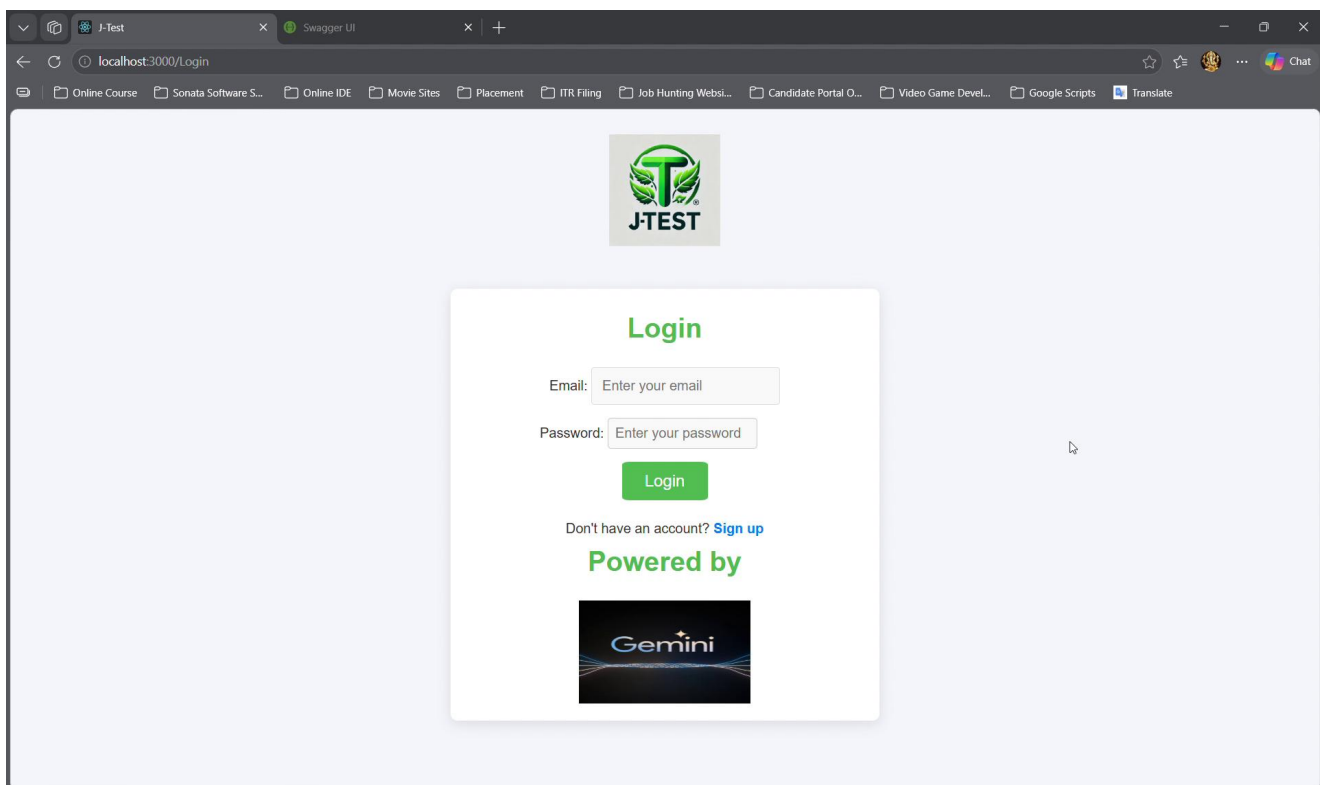


Figure 2 — Login page

6.2 Log in

1. Enter the registered email address.
2. Enter the password.
3. Select Login.
4. After successful authentication, J-Test stores the JWT in sessionStorage and opens the Home page.

Session behavior: The backend JWT lifetime is one hour. If protected API calls begin returning HTTP 401, sign in

again.

6.3 Navigate the application

After login, use the side-navigation arrow. Available destinations are Home, Chat, Settings, and Logout.

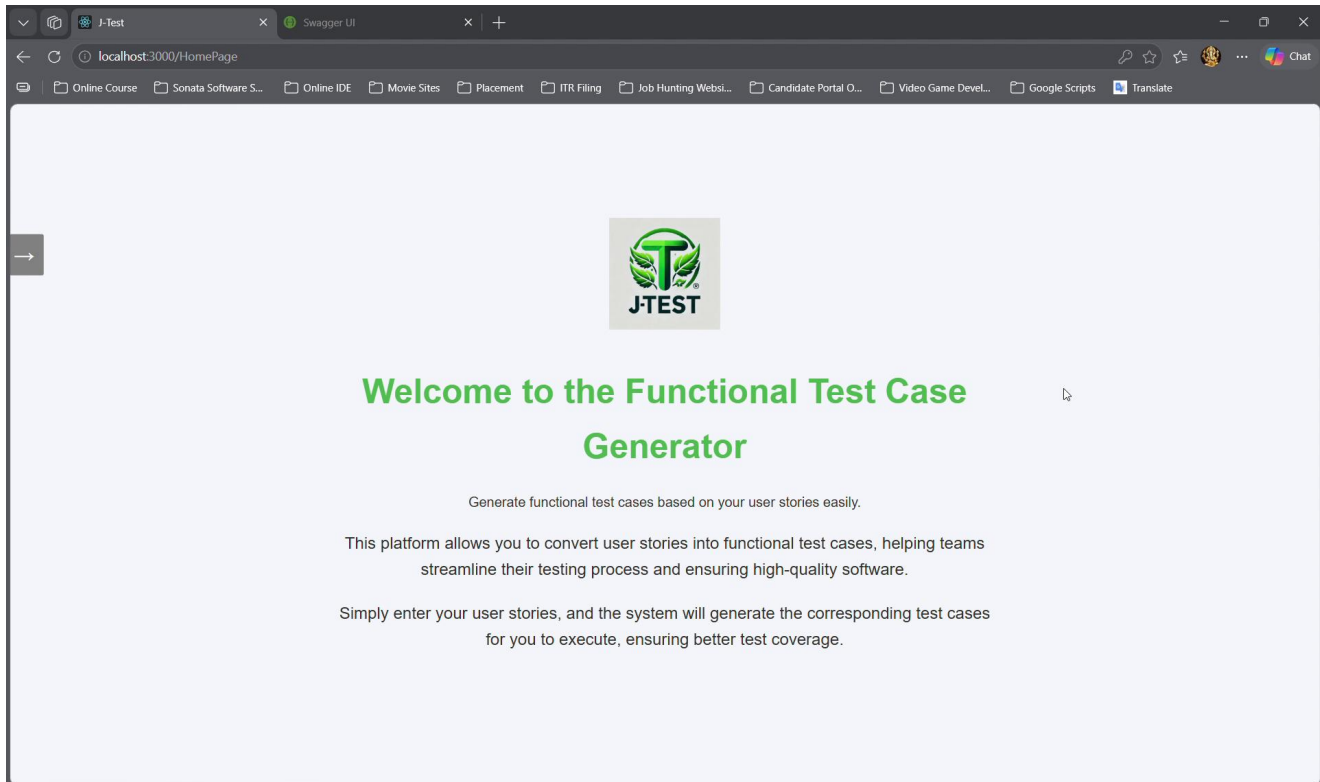


Figure 3 — Home page

6.4 Configure Jira credentials

Open Settings before importing Jira stories. The input area is initially locked; use the checkbox beside the action buttons to unlock the fields.

1. Open Settings from the side navigation.
2. Unlock the credential input fields.
3. Enter Domain as the Jira hostname only (for example, yourtenant.atlassian.net).
4. Enter Jira Username (typically the Jira/Atlassian account email).
5. Enter the Jira API Token.
6. Select Save. The API token is protected before being stored and is not returned to the browser.
7. Under Stored Credentials, select exactly one credential and choose Load to continue to Chat with that credential.

Figure 4 — Jira Settings screen

V2 credential handling: The Stored Credentials table shows “Stored securely” instead of the Jira token. Existing pre-V2/plaintext credentials should be deleted and recreated through the V2 Settings workflow.

6.5 Import a Jira user story

1. Load one saved Jira credential from Settings. J-Test navigates to Chat with that credential selected.
2. Open the Jira User Stories sidebar using the arrow on the right side of the Chat screen.
3. Confirm the Domain and Username are pre-populated and read-only.
4. Enter the Jira Project value used by your site (for example, a project key/name accepted by your Jira JQL).
5. Enter Issue Type, such as Story.
6. Select Fetch.
7. Review the returned story cards. Each card displays summary, assignee label, and description.
8. Select a story card to copy its description into the Chat input area.
9. Use Clear to clear fetched Jira results if needed.

Jira query behavior: The backend uses the saved credential ID, project, and issue type. The Jira API token stays on the server and is unprotected only when the backend prepares the outbound Jira request.

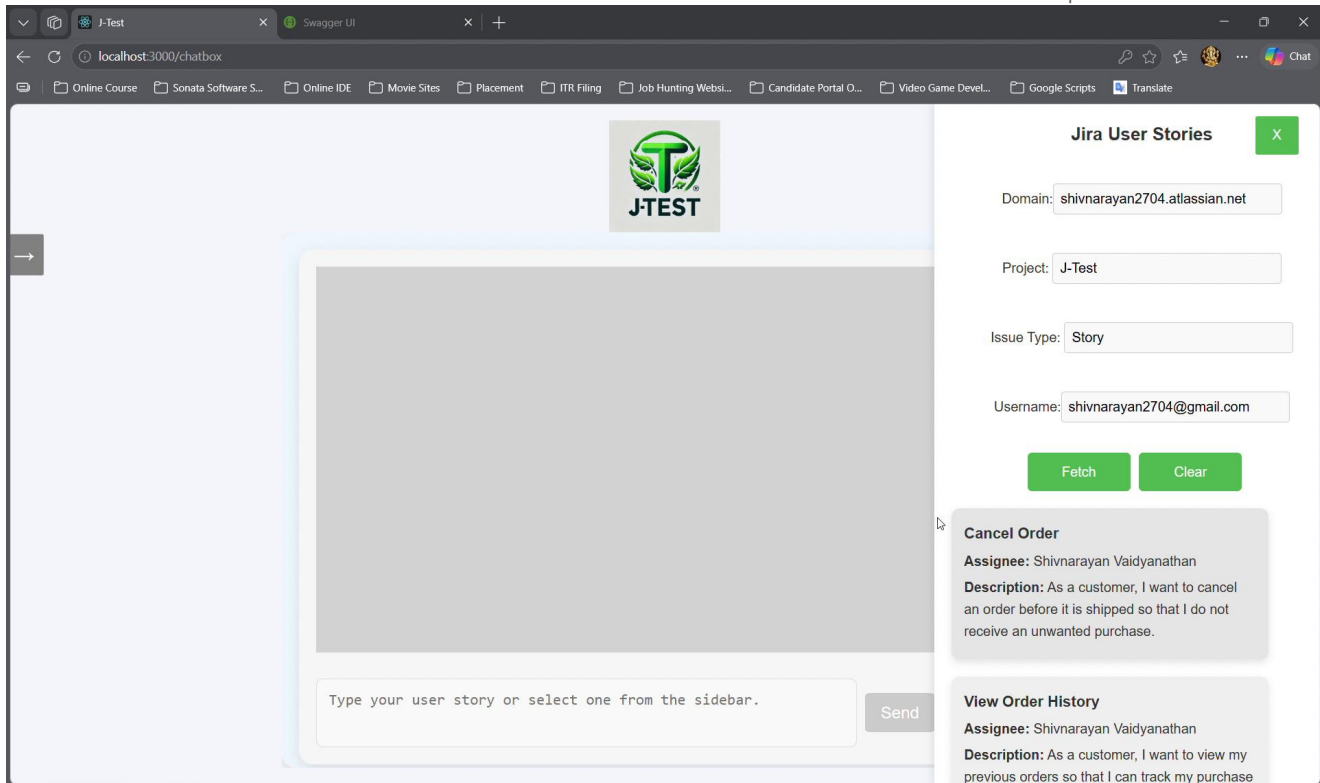


Figure 5 — Jira User Stories sidebar with fetched stories

6.6 Enter or select a user story

A story can be entered manually or selected from Jira. The application validates that the input resembles a user-story format before generation.



Figure 6 — User story selected in Chat

6.7 Generate functional test cases

1. Ensure the user-story text is present in the Chat input.
2. Select Send.
3. Wait for Gemini processing to complete.
4. Review the generated functional test cases in the chat response area. Markdown is converted to HTML and sanitized before it is rendered.

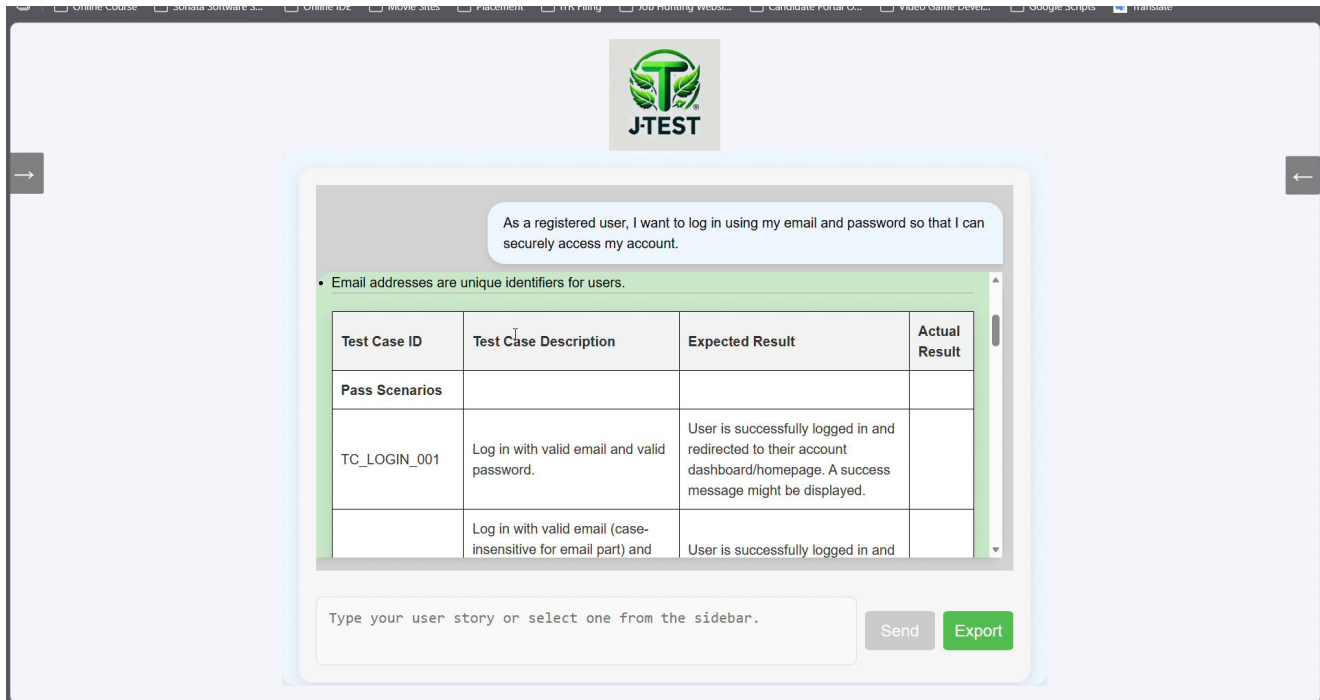


Figure 7 — Generated functional test cases

6.8 Export generated output to Word

After a bot/AI response exists, the Export button becomes available. Selecting Export converts the bot output to a Word document and downloads it as Functional Test Cases.docx.

1. Generate at least one AI response.
2. Select Export.
3. Open the downloaded Functional Test Cases.docx and review the generated content.

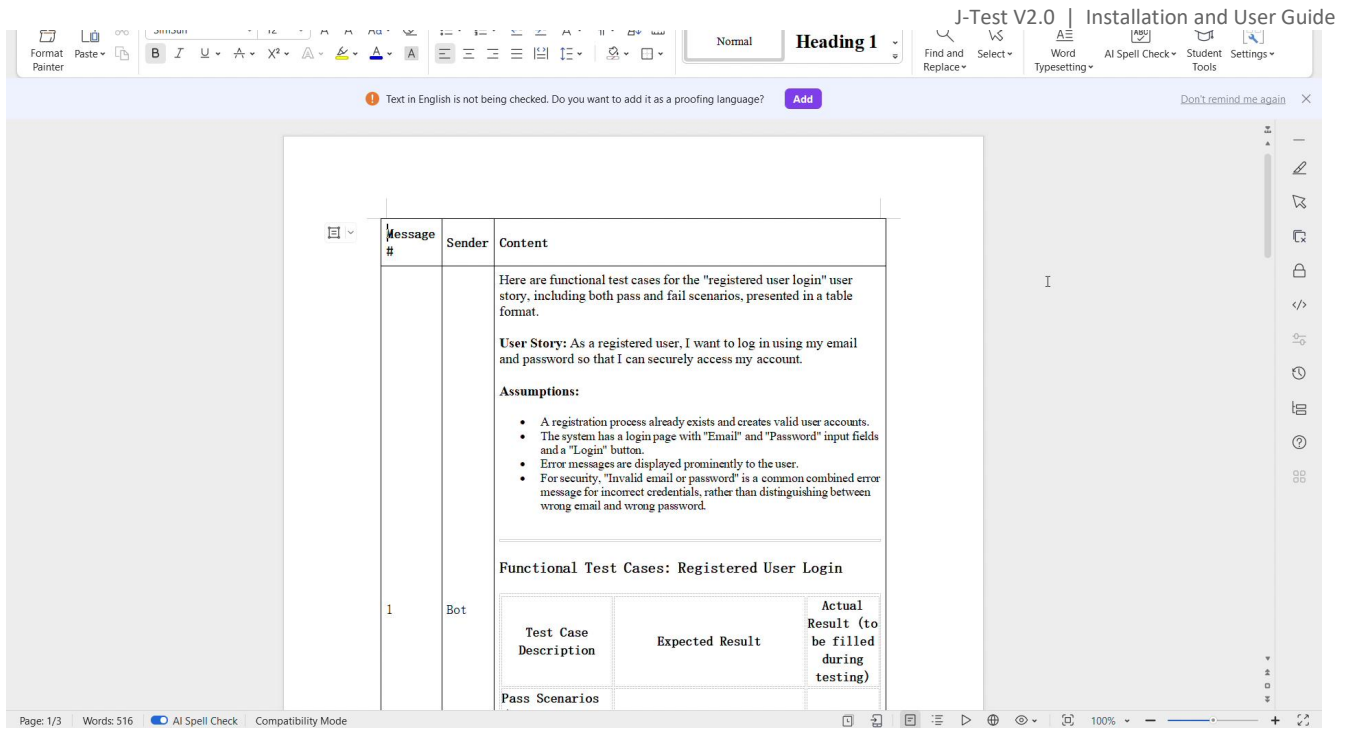


Figure 8 — Example exported Word document

6.9 Delete saved data

- Jira credential: in Settings > Stored Credentials, select the row and choose Delete.
- Application signup: in Settings > Stored Sign-ups, select the user row and choose Delete.

6.10 Log out

Open the side navigation and select Logout. The frontend removes authToken from sessionStorage and returns to Login.

7. Security behavior in V2

Control	V2 behavior
Password storage	Passwords are hashed using ASP.NET Core PasswordHasher; plaintext passwords are not returned by the credentials API.
Authentication	JWT Bearer authentication protects sensitive controllers/endpoints; Login and Signup are anonymous entry points.
JWT handling	Frontend stores the JWT in sessionStorage and sends Authorization: Bearer on protected requests.
Jira API tokens	Protected with ASP.NET Core Data Protection before database storage; never returned to the browser.
Jira SSRF protection	Outbound Jira host must match the configured allowlist and must be a hostname-only value.
XSS protection	User messages render as React text; AI HTML is sanitized with DOMPurify.

Error handling	Internal exception details are not returned to API clients.
Logging	Full Jira bodies, tokens, user stories, and detailed JWT exception messages are not logged.
Swagger	Available only when ASPNETCORE_ENVIRONMENT is Development.
Dependencies	V2 removed the vulnerable Jira.SDK/RestSharp chain and migrated the frontend build from Create React App/react-scripts to Vite.

8. Troubleshooting

Symptom	Likely cause / action
JWT signing key is not configured	Run dotnet user-secrets set "Jwt:Key" "<strong secret>" from the backend project folder.
Gemini API key is not configured	Set Gemini:ApiKey with User Secrets. Do not add the real key to appsettings.json.
Gemini request fails	Verify the API key is valid and Gemini:Model is available to that API key. V2 is configured for gemini-2.5-flash.
HTTP 401 on Settings/Jira/Generate	JWT is missing, malformed, or expired. Log in again and retry.
The configured Jira domain is not allowed	Add the exact Jira hostname to Jira:AllowedHosts. Do not include https://, paths, or ports.
Unable to unprotect Jira API token	The record may be legacy/plaintext or protected under a different Data Protection key context. Delete the saved Jira credential and recreate it through V2 Settings.
Jira returns HTTP 410 Gone	Confirm the current backend uses /rest/api/2/search/jql rather than the retired /rest/api/2/search endpoint.
Frontend cannot reach backend	Confirm backend is listening at http://localhost:5104 and frontend is running at http://localhost:3000.
LocalDB connection fails	Confirm MSSQLLocalDB is installed/running, J_Test_Credentials is attached, and the connection string catalog matches.
dotnet run blocked with 0x800711C7	This is a Windows Application Control/Code Integrity issue, not a C# build error. On managed devices use approved signing/allowlisting. On a personal device review Windows Security > App & browser control > Smart App Control and local policy before changing security settings.

9. Operational checklist

Check	Complete
.NET 8 SDK installed	☐
Node meets Vite engine requirement	☐
MSSQLLocalDB running	☐
J_Test_Credentials database attached	☐
Gemini:ApiKey configured in User Secrets	☐
Jwt:Key configured in User Secrets	☐
Jira hostname added to Jira:AllowedHosts	☐
dotnet build succeeds	☐
npm run build succeeds	☐
Backend starts on port 5104	☐
Frontend starts on port 3000	☐
Signup/Login works	☐
Jira credential can be saved and loaded	☐
Jira stories can be fetched	☐
Gemini test cases can be generated	☐
Export produces Functional Test Cases.docx	☐
Logout returns to Login	☐

Repository hygiene: Before publishing or sharing the repository, keep real secrets out of Git and consider excluding generated output such as bin, obj, node_modules, database working copies, and IDE metadata as appropriate for your delivery model.

End of Guide